

forecaster's toolbox

Guibril Ramde

Exercise 5

5.1 Produce forecasts for the following series using whichever of NAIVE(y), SNAIVE(y) or RW(y ~ drift()) is more appropriate in each case:

```
library(fpp3)
```

Warning: package 'fpp3' was built under R version 4.5.2

-- Attaching packages ----- fpp3 1.0.3 --

v tibble	3.3.1	v tsibble	1.2.0
v dplyr	1.2.0	v tsibbledata	0.4.1
v tidyr	1.3.2	v ggtime	1.0.0
v lubridate	1.9.4	v feasts	0.5.0
v ggplot2	4.0.2	v fable	0.5.0

Warning: package 'tibble' was built under R version 4.5.2

Warning: package 'dplyr' was built under R version 4.5.2

Warning: package 'tidyr' was built under R version 4.5.2

Warning: package 'ggplot2' was built under R version 4.5.2

Warning: package 'tsibble' was built under R version 4.5.2

Warning: package 'ggtime' was built under R version 4.5.2

Warning: package 'feasts' was built under R version 4.5.2

Warning: package 'fabletools' was built under R version 4.5.2

Warning: package 'fable' was built under R version 4.5.2

```
-- Conflicts ----- fpp3_conflicts --
x lubridate::date()      masks base::date()
x dplyr::filter()        masks stats::filter()
x tsibble::intersect()   masks base::intersect()
x tsibble::interval()    masks lubridate::interval()
x dplyr::lag()           masks stats::lag()
x tsibble::setdiff()     masks base::setdiff()
x tsibble::union()       masks base::union()
```

```
library(tsibble)
library(tidyverse)
```

Warning: package 'readr' was built under R version 4.5.2

Warning: package 'purrr' was built under R version 4.5.2

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v forcats 1.0.1      v readr  2.1.6
v purrr  1.2.1      v stringr 1.6.0
```

```
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter()      masks stats::filter()
x tsibble::interval()  masks lubridate::interval()
x dplyr::lag()          masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

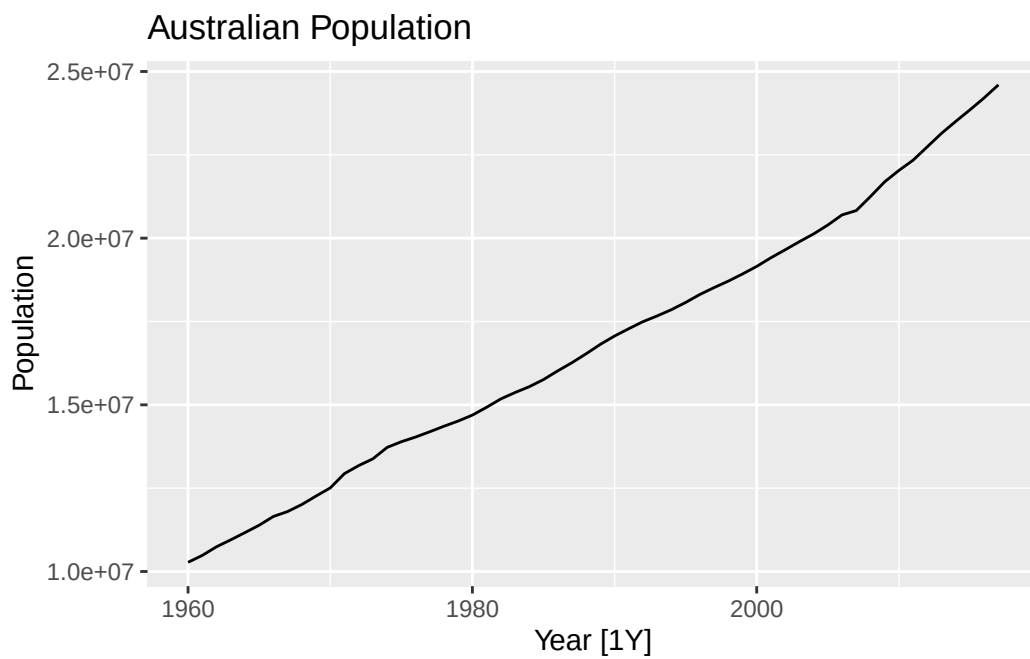
```
library(dplyr)
library(ggplot2)
#install.packages("fpp3")
library(ggtime)
```

Australian Population (global economy)

```

#Data preparation
aus_population <- global_economy |>
  filter(Country == "Australia")
# Plot visualization
aus_population |>
  autoplot(Population) +
  labs(
    y = "Population",
    title = "Australian Population"
  )

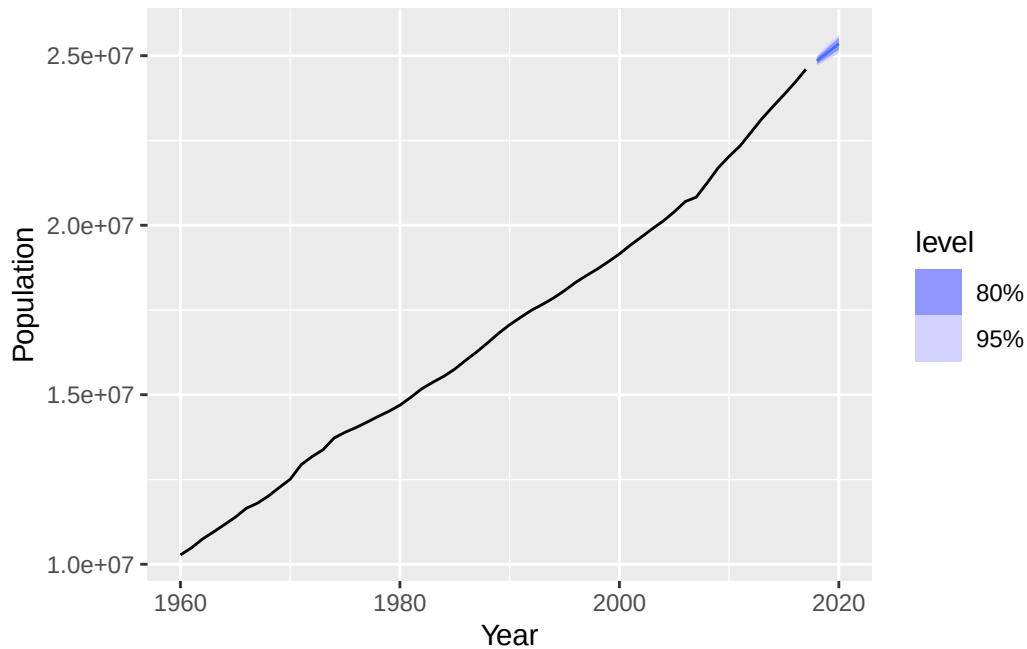
```



```

#Model selection
fit_population <- aus_population |>
  model(RW(Population ~ drift()))
#generate and forecast in 3 years
population_fc <- fit_population |>
  forecast(h = "3 years")
#plot the forecast
population_fc |>
  autoplot(aus_population)

```



Interpretation:

Australian population shows a clear upward trend and no seasonal pattern. Therefore, I selected the random walk with drift model.

#Bricks (aus_production)

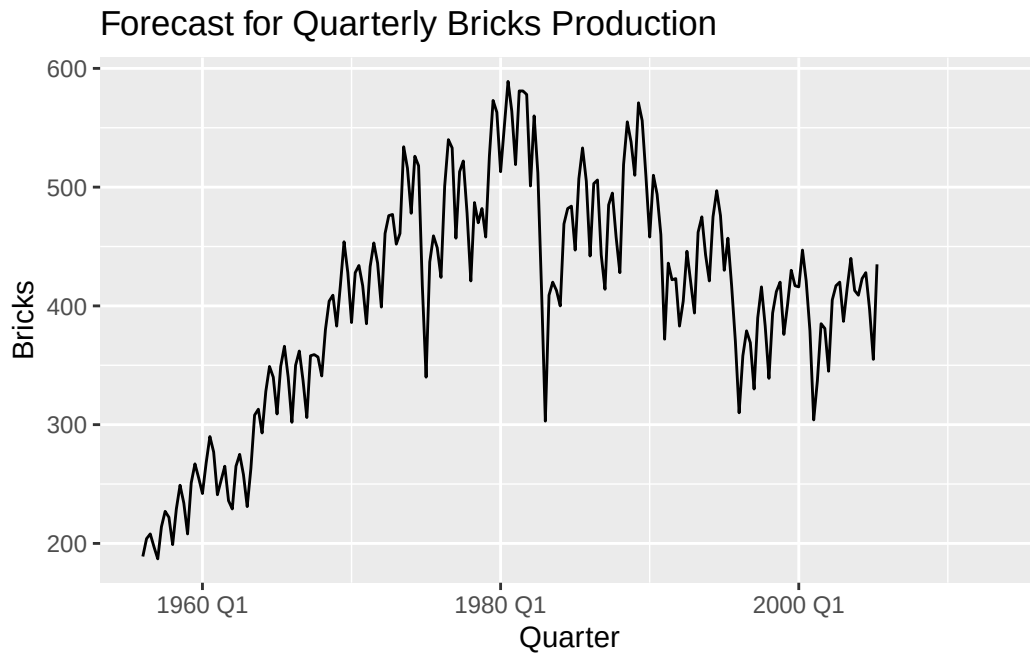
```
# Data preparation (tidy) and Plot (visualization)
aus_prod <- aus_production |>
  select(Bricks)
bricks_fit <- aus_prod |>
  model(SNAIVE(Bricks))

bricks_fc <- bricks_fit |>
  forecast(h = 12)

bricks_fc |>
  autoplot(aus_prod, level = NULL) +
  labs(
    y = "Bricks",
    title = "Forecast for Quarterly Bricks Production"
  )
```

Warning: Removed 12 rows containing missing values or values outside the scale range (``geom_line()``).

Warning: Removed 20 rows containing missing values or values outside the scale range (``geom_line()``).



Interpretation:

Since bricks is a quarterly data and a strong seasonality a SNAIVE is fit to use.

##NSW Lambs (aus_livestock)

```
aus_livestock |>
  distinct(State)
```

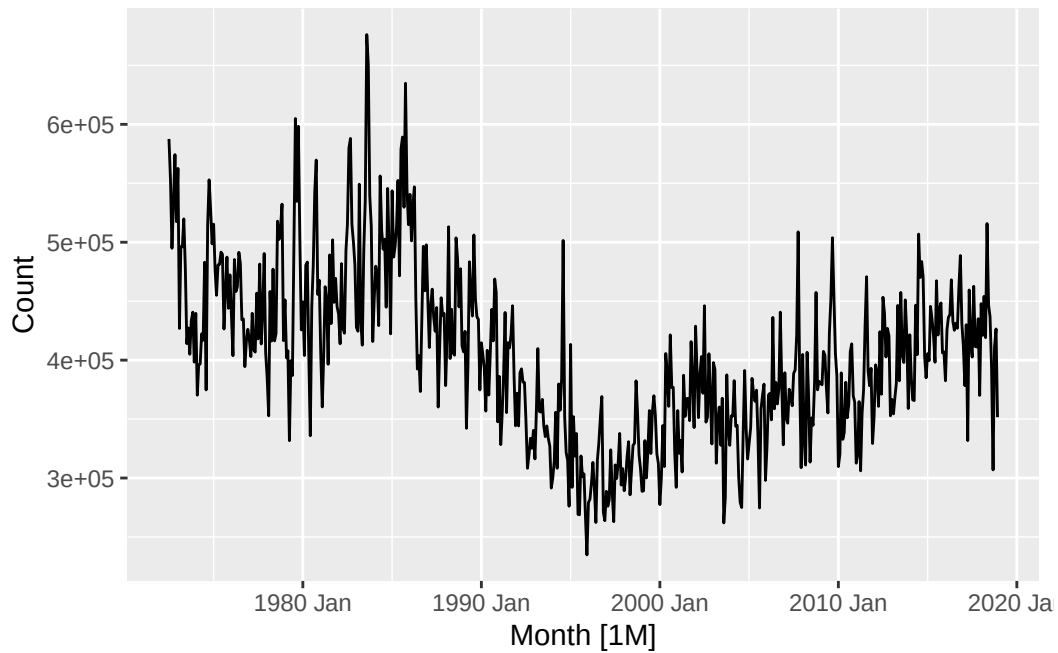
```
# A tibble: 8 x 1
  State
<fct>
1 Australian Capital Territory
2 New South Wales
3 Northern Territory
```

```
4 Queensland
5 South Australia
6 Tasmania
7 Victoria
8 Western Australia
```

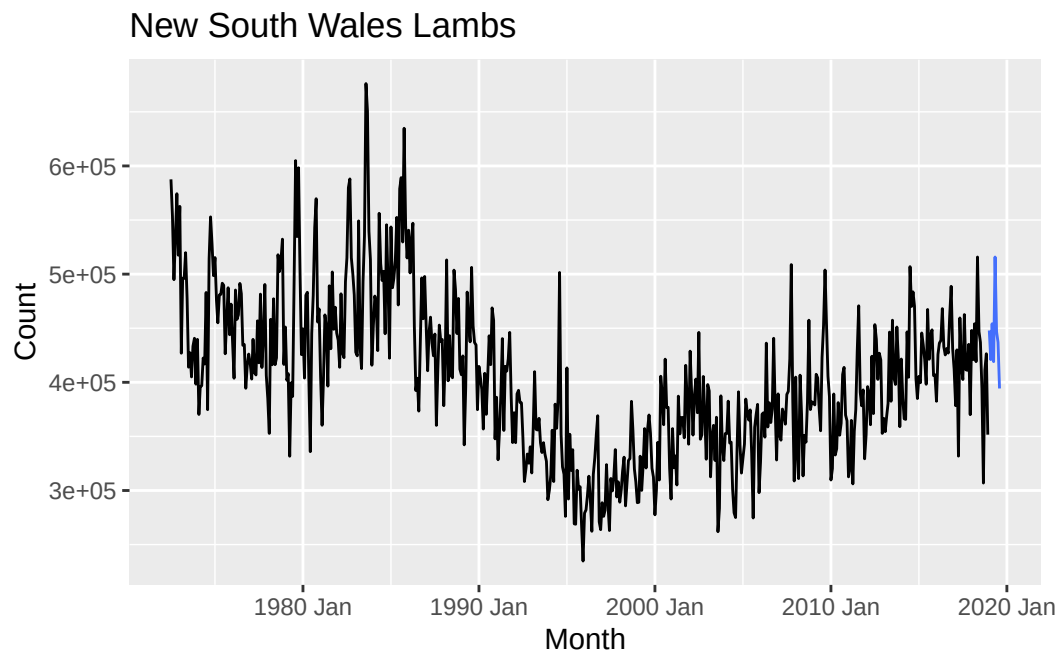
```
nsw_lambs <- aus_livestock |>
  filter(State == "New South Wales", Animal == "Lambs")
nsw_lambs
```

```
# A tsibble: 558 x 4 [1M]
# Key:       Animal, State [1]
   Month Animal State      Count
   <month> <fct>  <fct>      <dbl>
1 1972 Jul  Lambs  New South Wales 587600
2 1972 Aug  Lambs  New South Wales 553700
3 1972 Sep  Lambs  New South Wales 494900
4 1972 Oct  Lambs  New South Wales 533500
5 1972 Nov  Lambs  New South Wales 574300
6 1972 Dec  Lambs  New South Wales 517500
7 1973 Jan  Lambs  New South Wales 562600
8 1973 Feb  Lambs  New South Wales 426900
9 1973 Mar  Lambs  New South Wales 496300
10 1973 Apr Lambs  New South Wales 496000
# i 548 more rows
```

```
nsw_lambs |>
  autoplot(Count)
```



```
#Model selection
fit_nsw <- nsw_lambs |> model(SNAIVE(Count))
#Generate forecast
nsw_fit <- fit_nsw |> forecast(h = 8)
#Plot forecast
nsw_fit |>
  autoplot(nsw_lambs, level = NULL) +
  #autolayer(filter_index(nsw_lambs, "2020" ~ .), colour = "red") +
  labs(title = "New South Wales Lambs")
```

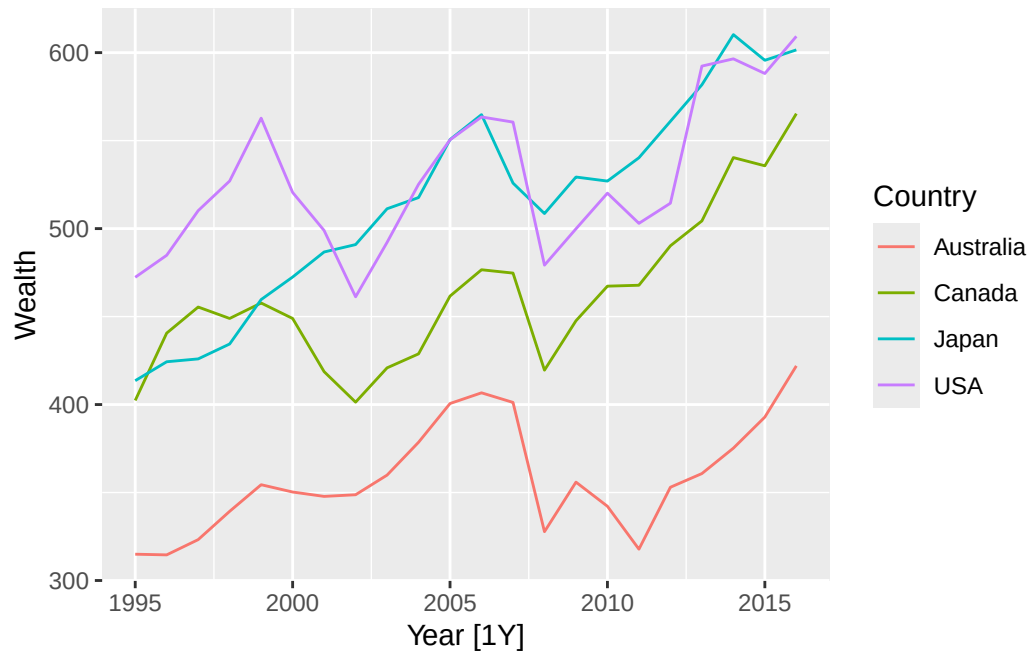


Interpretation:

The NSW Lambs series shows a seasonal pattern, so the seasonal naïve method is appropriate.

Household Wealth

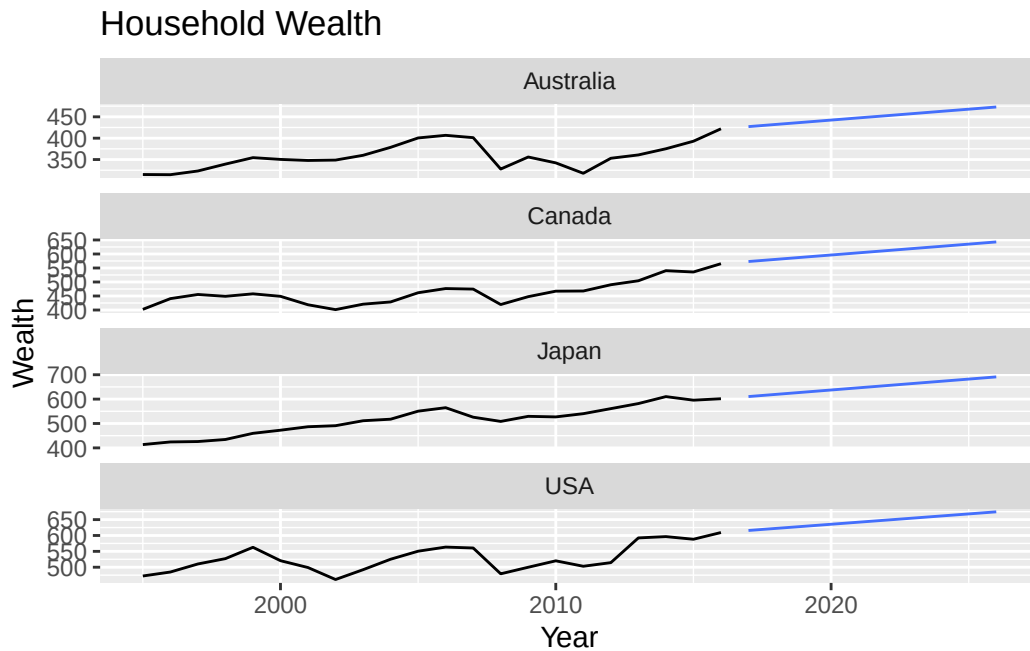
```
hh_budget |>  
  autoplot(Wealth)
```



```
fit_house <- hh_budget |>
  model(RW(Wealth ~ drift()))

house_fc <- fit_house |>
  forecast(h = 10)

house_fc |>
  autoplot(hh_budget, level = NULL) +
  labs(title = "Household Wealth")
```



Interpretation:

RW model is good fit for the forecast of this datasets because the models makes sense for a series dominated by a trend without a strong seasonal pattern.

Retail

```
aus_retail |>
  distinct(State)
```

```
# A tibble: 8 x 1
  State
  <chr>
1 Australian Capital Territory
2 New South Wales
3 Northern Territory
4 Queensland
5 South Australia
6 Tasmania
7 Victoria
```

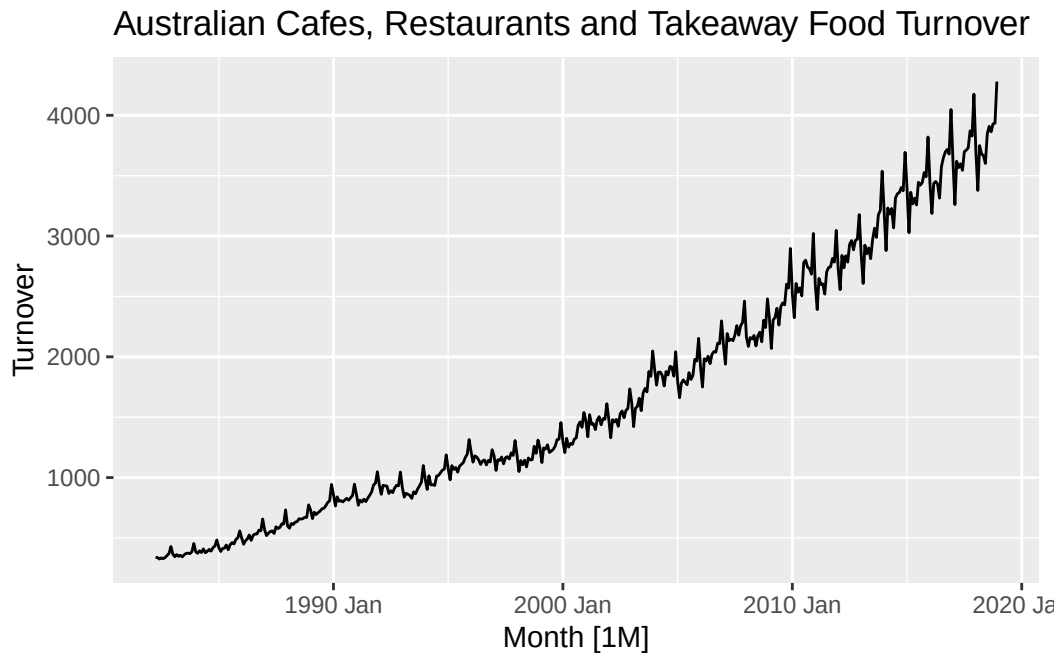
8 Western Australia

```
aus_retail |>
  distinct(Industry)
```

```
# A tibble: 20 x 1
  Industry
  <chr>
1 Cafes, restaurants and catering services
2 Cafes, restaurants and takeaway food services
3 Clothing retailing
4 Clothing, footwear and personal accessory retailing
5 Department stores
6 Electrical and electronic goods retailing
7 Food retailing
8 Footwear and other personal accessory retailing
9 Furniture, floor coverings, houseware and textile goods retailing
10 Hardware, building and garden supplies retailing
11 Household goods retailing
12 Liquor retailing
13 Newspaper and book retailing
14 Other recreational goods retailing
15 Other retailing
16 Other retailing n.e.c.
17 Other specialised food retailing
18 Pharmaceutical, cosmetic and toiletry goods retailing
19 Supermarket and grocery stores
20 Takeaway food services
```

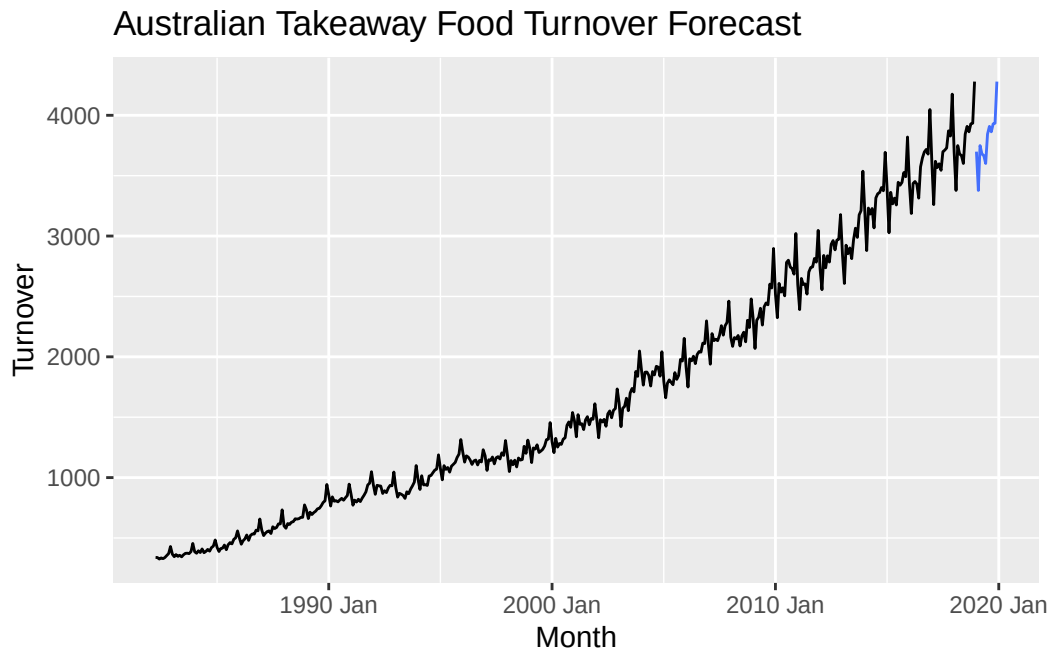
```
food_turnover <- aus_retail |>
  filter(Industry == "Cafes, restaurants and takeaway food services") |>
  index_by(Month) |>
  summarise(Turnover = sum(Turnover))

food_turnover |>
  autoplot(Turnover) +
  labs(
    title = "Australian Cafes, Restaurants and Takeaway Food Turnover",
    y = "Turnover"
  )
```



```
# Model selection
fit_food <- food_turnover |> model(SNAIVE(Turnover))

# Generate forecast
fit_fd <- fit_food |> forecast(h = 12)
fit_fd |>
  autoplot(food_turnover, level = NULL ) +
  #autolayer(filter_index(aus_retail, '2020' ~ .), colour = 'red') +
  labs(
    title = "Australian Takeaway Food Turnover Forecast",
    y = "Turnover"
  )
```



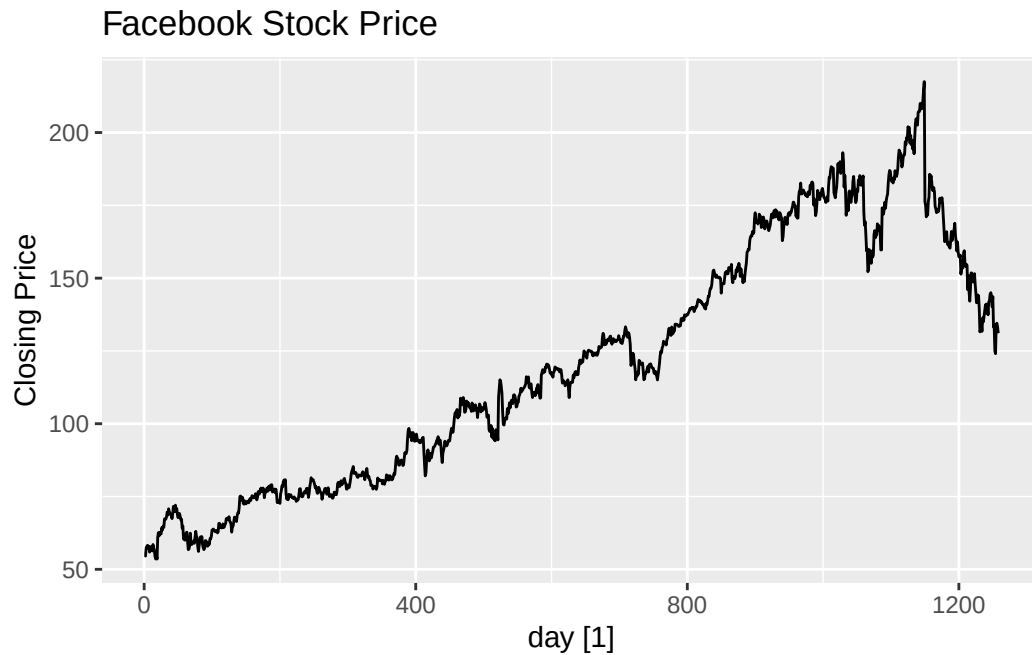
Interpretation:

The Australian takeaway food turnover series shows a recurring seasonal pattern. Therefore, I selected the seasonal naïve method, which uses the corresponding observation from the previous year to forecast each month.

5.2 Use the Facebook stock price (data set `gafa_stock`) to do the following:

- Produce a time plot of the series

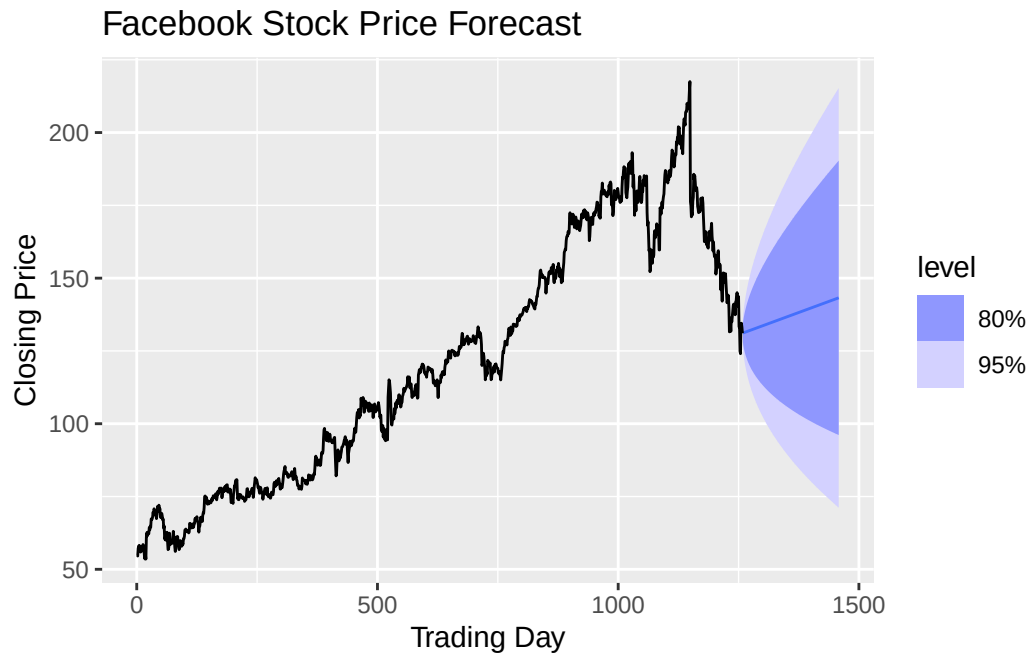
```
fb_data <- gafa_stock |>
  filter(Symbol == "FB") |>
  mutate(day = row_number()) |>
  update_tsibble(index = day, regular = TRUE)
fb_data |>
  autoplot(Close) +
  labs(
    title = "Facebook Stock Price",
    y = "Closing Price"
  )
```



```
#b Produce Forecast using drift
fit_fb <- fb_data |> model(RW(Close ~ drift()))
fit_fb
```

```
# A mable: 1 x 2
# Key:      Symbol [1]
# Symbol `RW(Close ~ drift())`
#   <chr>          <model>
# 1 FB            <RW w/ drift>
```

```
#forecast
fb_fit <- fit_fb |> forecast(h = 200)
#Plot the forecast
fb_fit |>
  autoplot(fb_data) +
  labs(
    title = "Facebook Stock Price Forecast",
    y = "Closing Price",
    x = "Trading Day"
  )
```



C. Show that the forecasts are identical to extending the line drawn between the first and last observations.

```
first_price <- first(fb_data$Close)
first_price
```

```
[1] 54.71
```

```
last_price <- last(fb_data$Close)
last_price
```

```
[1] 131.09
```

```
n <- nrow(fb_data)
n
```

```
[1] 1258
```

```
#Calculating the slop
slope <- (last_price - first_price)/(n - 1)
slope
```

```
[1] 0.06076372
```

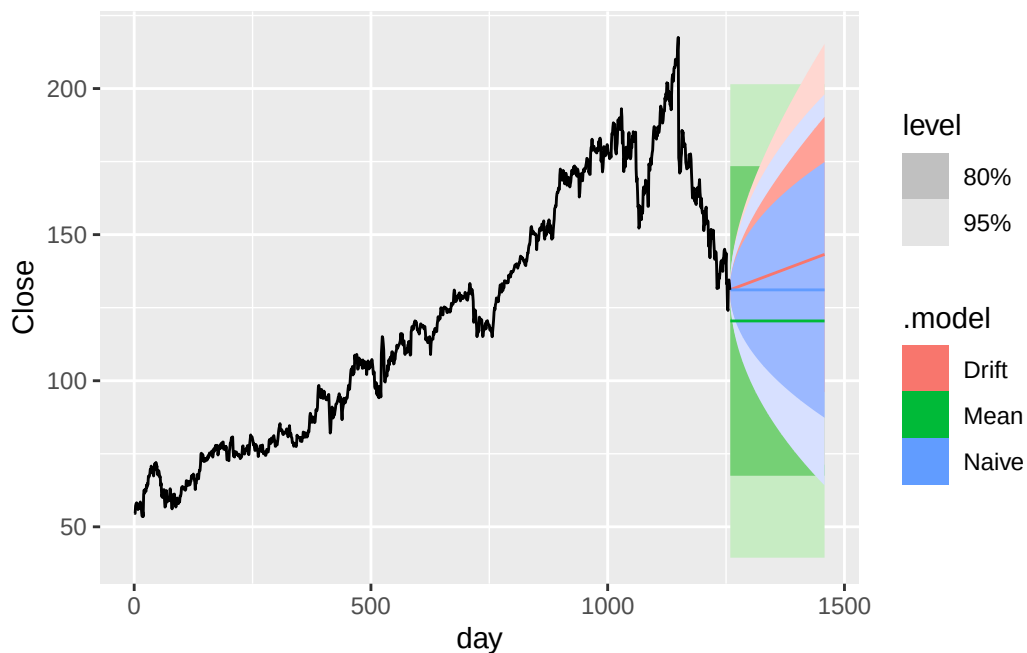
```
##Calculating future values
future_days <- 1:200

extend_line_price <- last_price + future_days * slope
extend_line_price
```

```
[1] 131.1508 131.2115 131.2723 131.3331 131.3938 131.4546 131.5153 131.5761
[9] 131.6369 131.6976 131.7584 131.8192 131.8799 131.9407 132.0015 132.0622
[17] 132.1230 132.1837 132.2445 132.3053 132.3660 132.4268 132.4876 132.5483
[25] 132.6091 132.6699 132.7306 132.7914 132.8521 132.9129 132.9737 133.0344
[33] 133.0952 133.1560 133.2167 133.2775 133.3383 133.3990 133.4598 133.5205
[41] 133.5813 133.6421 133.7028 133.7636 133.8244 133.8851 133.9459 134.0067
[49] 134.0674 134.1282 134.1889 134.2497 134.3105 134.3712 134.4320 134.4928
[57] 134.5535 134.6143 134.6751 134.7358 134.7966 134.8573 134.9181 134.9789
[65] 135.0396 135.1004 135.1612 135.2219 135.2827 135.3435 135.4042 135.4650
[73] 135.5257 135.5865 135.6473 135.7080 135.7688 135.8296 135.8903 135.9511
[81] 136.0119 136.0726 136.1334 136.1941 136.2549 136.3157 136.3764 136.4372
[89] 136.4980 136.5587 136.6195 136.6803 136.7410 136.8018 136.8625 136.9233
[97] 136.9841 137.0448 137.1056 137.1664 137.2271 137.2879 137.3487 137.4094
[105] 137.4702 137.5310 137.5917 137.6525 137.7132 137.7740 137.8348 137.8955
[113] 137.9563 138.0171 138.0778 138.1386 138.1994 138.2601 138.3209 138.3816
[121] 138.4424 138.5032 138.5639 138.6247 138.6855 138.7462 138.8070 138.8678
[129] 138.9285 138.9893 139.0500 139.1108 139.1716 139.2323 139.2931 139.3539
[137] 139.4146 139.4754 139.5362 139.5969 139.6577 139.7184 139.7792 139.8400
[145] 139.9007 139.9615 140.0223 140.0830 140.1438 140.2046 140.2653 140.3261
[153] 140.3868 140.4476 140.5084 140.5691 140.6299 140.6907 140.7514 140.8122
[161] 140.8730 140.9337 140.9945 141.0552 141.1160 141.1768 141.2375 141.2983
[169] 141.3591 141.4198 141.4806 141.5414 141.6021 141.6629 141.7236 141.7844
[177] 141.8452 141.9059 141.9667 142.0275 142.0882 142.1490 142.2098 142.2705
[185] 142.3313 142.3920 142.4528 142.5136 142.5743 142.6351 142.6959 142.7566
[193] 142.8174 142.8782 142.9389 142.9997 143.0604 143.1212 143.1820 143.2427
```

Part D Try using some of the other benchmark functions to forecast the same data set. Which do you think is best? Why?

```
fb_fit <- fb_data |>
  model(
    Naive = NAIVE(Close),
    Drift = RW(Close ~ drift()),
    Mean = MEAN(Close)
  )
#Forecast
forecast_fb <- fb_fit |> forecast(h = 200)
forecast_fb |>
  autoplot(fb_data)
```



Interpretation:

Looking at the forecasts, I think the drift method is more appropriate. The Facebook stock price shows an overall increasing trend over the observed period. The mean method does not account for this trend, while the naïve method assumes that future prices will remain at the last observed value. The drift method accounts for the overall upward movement by extending the average change between the first and last observations.

5.3. Apply a seasonal naïve method to the quarterly Australian beer production data from 1992. Check if the residuals look like white noise, and plot the forecasts. The following code will help.

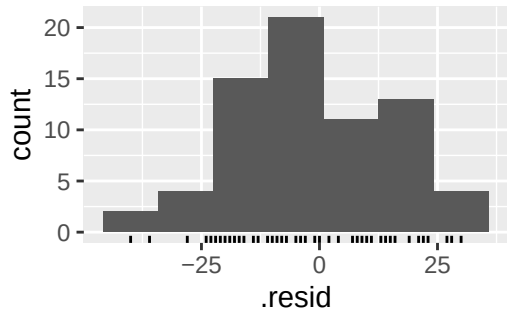
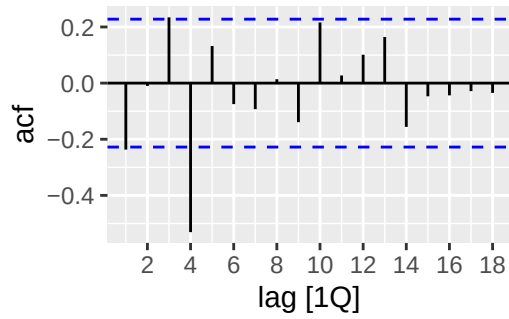
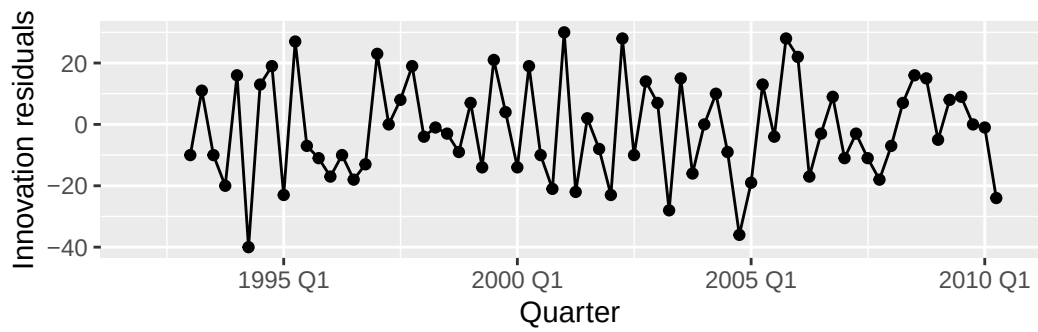
```
# Extract data of interest
recent_production <- aus_production |>
  filter(year(Quarter) >= 1992)
# Define and estimate a model
fit <- recent_production |> model(SNAIVE(Beer))
# Look at the residuals
fit |> gg_tsresiduals()
```

Warning: Removed 4 rows containing missing values or values outside the scale range (``geom_line()``).

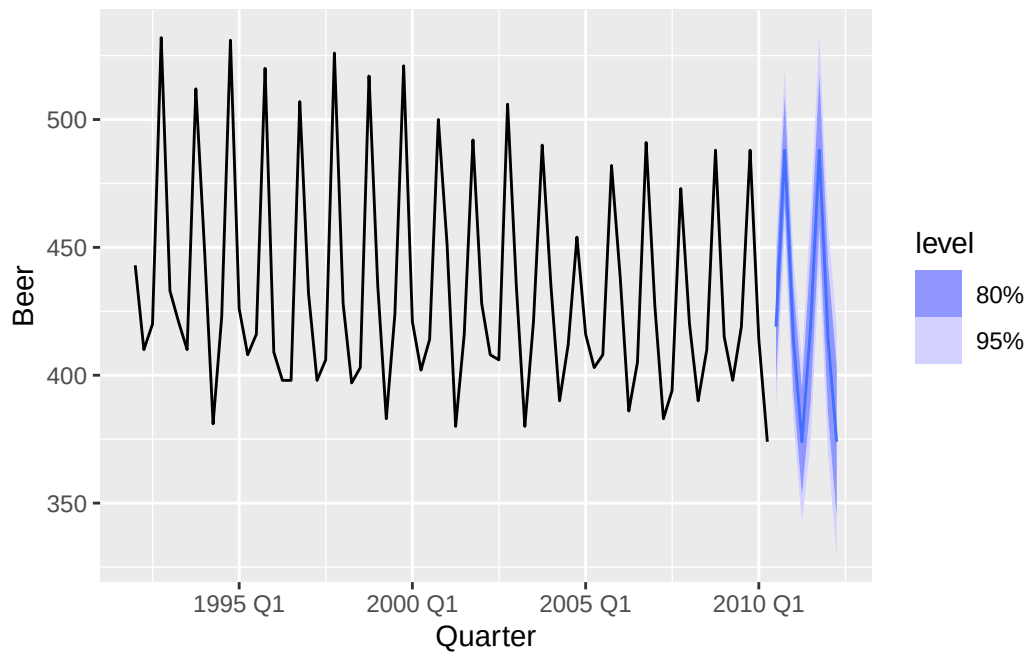
Warning: Removed 4 rows containing missing values or values outside the scale range (``geom_point()``).

Warning: Removed 4 rows containing non-finite outside the scale range (``stat_bin()``).

Warning: Removed 4 rows containing missing values or values outside the scale range (``geom_rug()``).



```
# Look a some forecasts
fit |> forecast() |> autoplot(recent_production)
```



Interpretation:

The residuals do not appear to be completely white noise. The ACF plot shows a significant negative spike at lag 4 and a positive spike around lag 3 that exceeds the significance bounds. Since the data are quarterly, lag 4 represents one year, suggesting that some seasonal information remains in the residuals. Therefore, the seasonal naïve model captures the general seasonal pattern, but it does not capture all of the information in the series.

5.4 Repeat the previous exercise using the Australian Exports series from `global_economy` and the Bricks series from `aus_production`. Use whichever of `NAIVE()` or `SNAIVE()` is more appropriate in each case.

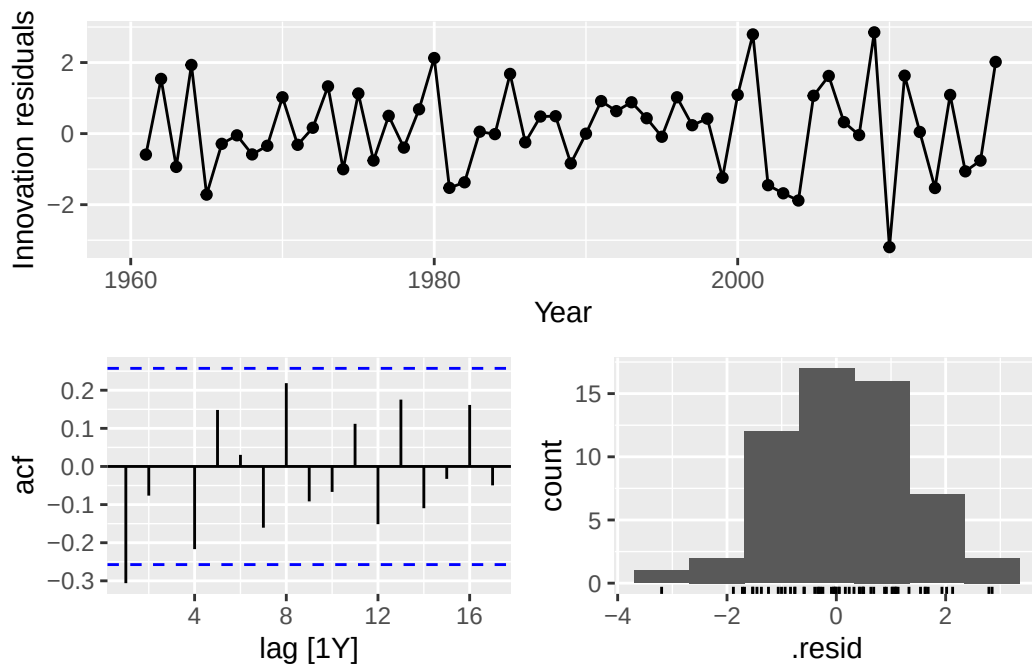
```
# Extract data of interest
australian_export <- global_economy |>
  filter(Country == "Australia")
# Define and estimate a model
fit <- australian_export |> model(NAIVE(Exports))
# Look at the residuals
fit |> gg_tsresiduals()
```

Warning: Removed 1 row containing missing values or values outside the scale range (``geom_line()``).

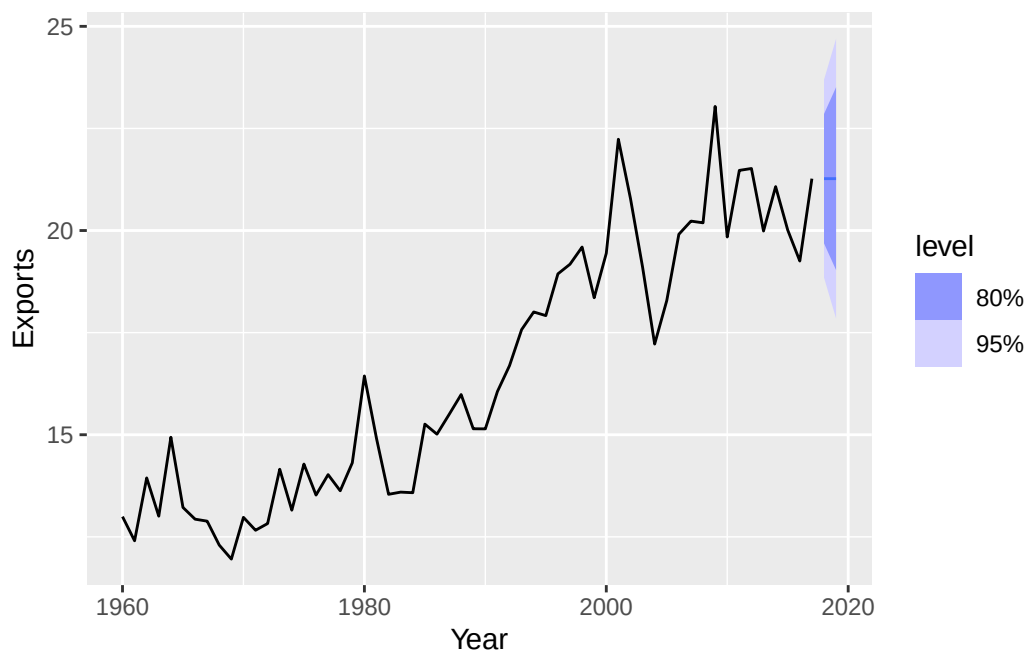
Warning: Removed 1 row containing missing values or values outside the scale range (``geom_point()``).

Warning: Removed 1 row containing non-finite outside the scale range (``stat_bin()``).

Warning: Removed 1 row containing missing values or values outside the scale range (``geom_rug()``).



```
# Look a some forecasts
fit |> forecast() |> autoplot(australian_export)
```



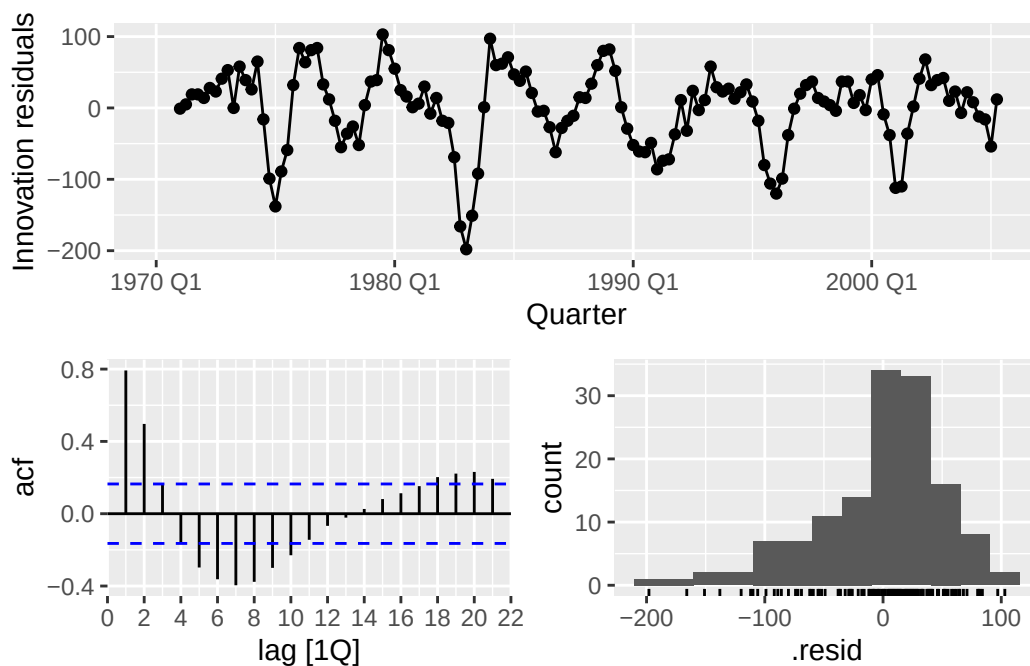
```
# Extract data of interest
australian_bricks <- aus_production |>
  filter(year(Quarter) >= 1970) |>
  filter(!is.na(Bricks))
# Define and estimate a model
fit_brick <- australian_bricks |> model(SNAIVE(Bricks))
# Look at the residuals
fit_brick |> gg_tsresiduals()
```

Warning: Removed 4 rows containing missing values or values outside the scale range (``geom_line()``).

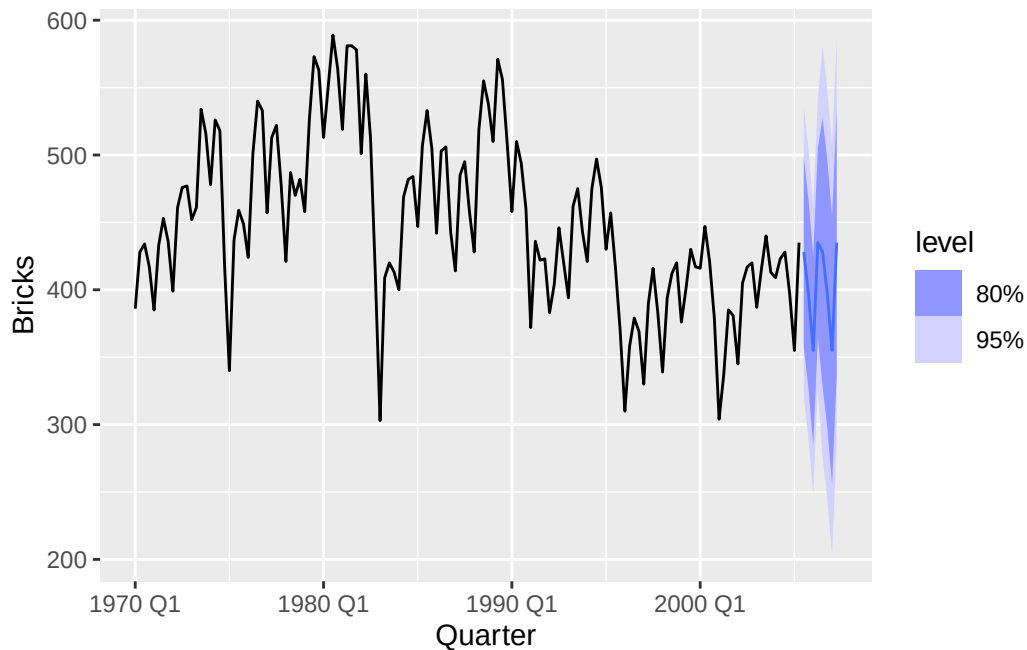
Warning: Removed 4 rows containing missing values or values outside the scale range (``geom_point()``).

Warning: Removed 4 rows containing non-finite outside the scale range (``stat_bin()``).

Warning: Removed 4 rows containing missing values or values outside the scale range (``geom_rug()``).



```
# Look at some forecasts  
fit_brick |> forecast() |> autoplot(australian_bricks)
```



Interpretation:

For the Australian Exports series, I used the naïve method because the data are annual and do not have a seasonal pattern. The naïve method forecasts future exports using the most recent observed value. For the Bricks series, I used the seasonal naïve method because the quarterly data show a recurring seasonal pattern. The seasonal naïve method forecasts each quarter using the value from the corresponding quarter of the previous year.

The residuals for both datasets do not appear to be completely white noise. For Australian Exports, the ACF plot shows a significant negative spike at lag 1, indicating that some autocorrelation remains in the residuals. For Bricks production, the ACF plot shows several significant positive and negative spikes that exceed the significance bounds. This indicates that there is still autocorrelation in the residuals. Therefore, although the seasonal naïve method captures the general seasonal pattern in Bricks production, it does not capture all of the information in the series.

5.7 For your retail time series (from Exercise 7 in Section 2.10):

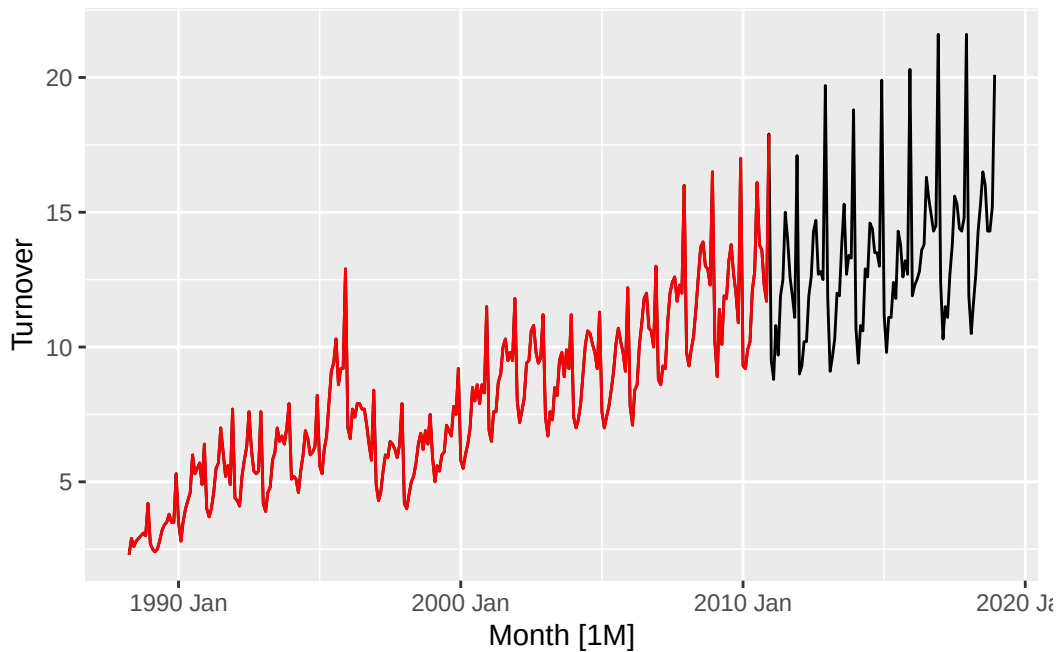
```

set.seed(12345678)
myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`,1))

myseries_train <- myseries |>
  filter(year(Month) < 2011)

autoplot(myseries, Turnover) +
  autolayer(myseries_train, Turnover, colour = "red")

```



```

fit <- myseries_train |>
  model(SNAIVE(Turnover))

fit |> gg_tsresiduals()

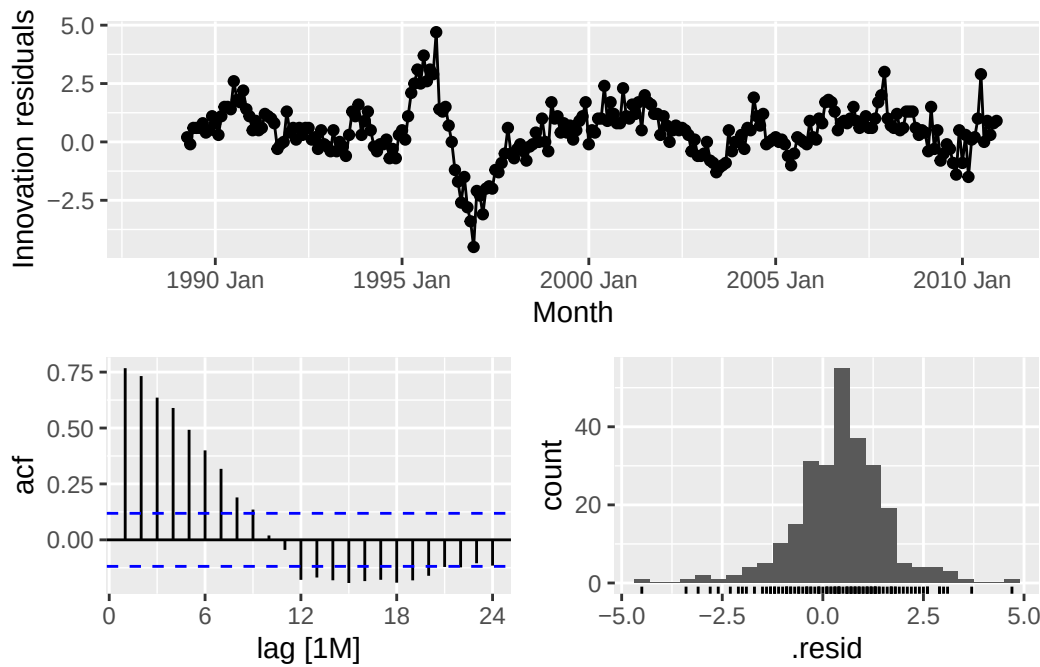
```

Warning: Removed 12 rows containing missing values or values outside the scale range (`geom\_line()`).

Warning: Removed 12 rows containing missing values or values outside the scale range (`geom\_point()`).

Warning: Removed 12 rows containing non-finite outside the scale range
(`stat\_bin()`).

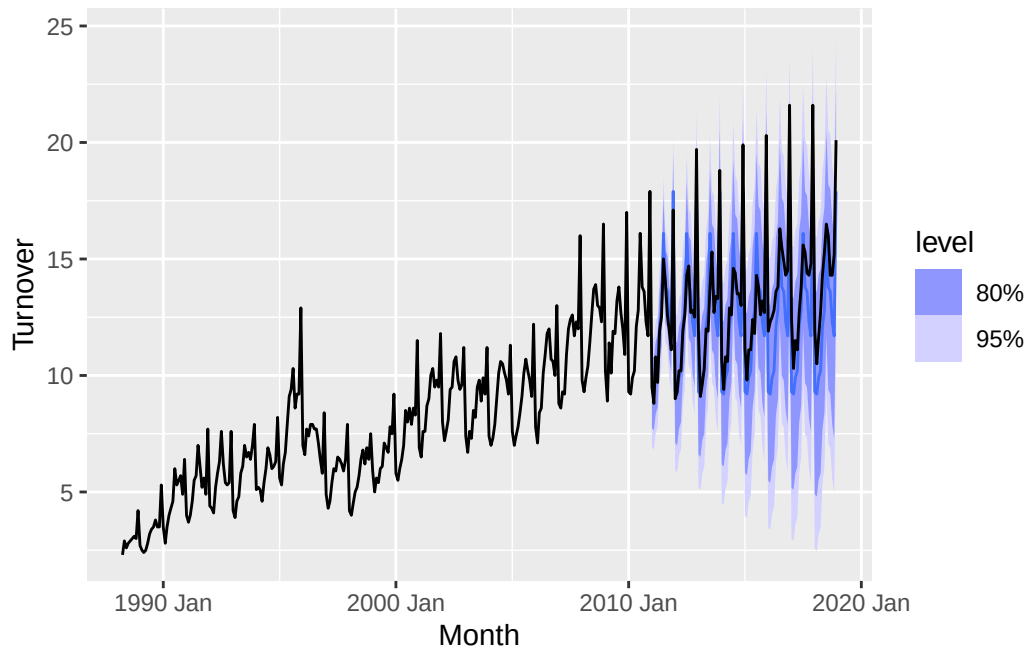
Warning: Removed 12 rows containing missing values or values outside the scale range
(`geom\_rug()`).



```
fc <- fit |>  
  forecast(new_data = anti_join(myseries, myseries_train))
```

Joining with `by = join\_by(State, Industry, `Series ID`, Month, Turnover)`

```
fc |> autoplot(myseries)
```



```
fit |> accuracy()
```

```
# A tibble: 1 x 12
```

	State	Industry	.model	.type	ME	RMSE	MAE	MPE	MAPE	MASE	RMSSE	ACF1
	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Norther~	Clothin~	SNAIV~	Trai~	0.439	1.21	0.915	5.23	12.4	1	1	0.768

```
fc |> accuracy(myseries)
```

```
# A tibble: 1 x 12
```

	.model	State	Industry	.type	ME	RMSE	MAE	MPE	MAPE	MASE	RMSSE	ACF1
	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	SNAIVE(T~	Nort~	Clothin~	Test	0.836	1.55	1.24	5.94	9.06	1.36	1.28	0.601

g. How sensitive are the accuracy measures to the amount of training data used?

```
train1 <- myseries |>
  filter(year(Month) >= 1990, year(Month) < 2011)
```

```

train2 <- myseries |>
  filter(year(Month) >= 2000, year(Month) < 2011)

train3 <- myseries |>
  filter(year(Month) >= 2005, year(Month) < 2011)

fit1 <- train1 |>
  model(SNAIVE(Turnover))

fit2 <- train2 |>
  model(SNAIVE(Turnover))

fit3 <- train3 |>
  model(SNAIVE(Turnover))

test <- myseries |>
  filter(year(Month) >= 2011)

fc1 <- fit1 |> forecast(new_data = test)
fc2 <- fit2 |> forecast(new_data = test)
fc3 <- fit3 |> forecast(new_data = test)

fc1 |> accuracy(myseries)

```

```

# A tibble: 1 x 12
  .model      State Industry .type      ME  RMSE  MAE  MPE  MAPE  MASE  RMSSE  ACF1
  <chr>      <chr> <chr>    <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 SNAIVE(T~ Nort~ Clothi~ Test  0.836  1.55  1.24  5.94  9.06  1.36  1.28  0.601

```

```

fc2 |> accuracy(myseries)

```

```

# A tibble: 1 x 12
  .model      State Industry .type      ME  RMSE  MAE  MPE  MAPE  MASE  RMSSE  ACF1
  <chr>      <chr> <chr>    <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 SNAIVE(T~ Nort~ Clothi~ Test  0.836  1.55  1.24  5.94  9.06  1.36  1.28  0.601

```

```
fc3 |> accuracy(myseries)
```

```
# A tibble: 1 x 12
```

	.model	State	Industry	.type	ME	RMSE	MAE	MPE	MAPE	MASE	RMSSE	ACF1
	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	SNAIVE(T~	Nort~	Clothin~	Test	0.836	1.55	1.24	5.94	9.06	1.36	1.28	0.601

```
#range(myseries$Month)
```

Interpretation:

I tested the seasonal naïve model using different amounts of training data by starting the training period in 1990, 2000, and 2005, while keeping the end of the training period at 2010. The test accuracy measures were identical for all three training sets, with an RMSE of 1.552, MAE of 1.241, and MAPE of 9.064%. Therefore, the forecast accuracy of the seasonal naïve method is not sensitive to the amount of older training data used in this case. This occurs because the seasonal naïve method bases its forecasts on the most recent seasonal observations, which are the same for all three training sets.